

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 23

Performance & Load Testing

k6 • Load Test Design • Resilience Under Real Load • Bottleneck Analysis

1. What You Will Learn

By the end of this session, you should be able to:

- Understand what a Virtual User (VU) means in k6.
 - Write and execute a basic k6 test.
 - Understand iterations and VUs.
 - Use checks to validate responses.
 - Use thresholds to define performance requirements.
 - Distinguish smoke, load, stress, and spike testing.
 - Design a realistic staged workload.
 - Test the complete secured microservices platform.
 - Observe latency, throughput, and error rate.
 - Correlate performance problems with application behavior.
 - Observe Resilience4j Bulkhead and Circuit Breaker behavior.
 - Use Zipkin to investigate slow requests.
 - Identify likely performance bottlenecks.
 - Understand why increasing VUs is not the same as simulating real users.
 - Understand why a performance test can fail because of the load generator itself.
-

2. Why Performance Testing?

Until now, we have tested our microservices mostly with:

```
curl  
Postman  
unit tests
```

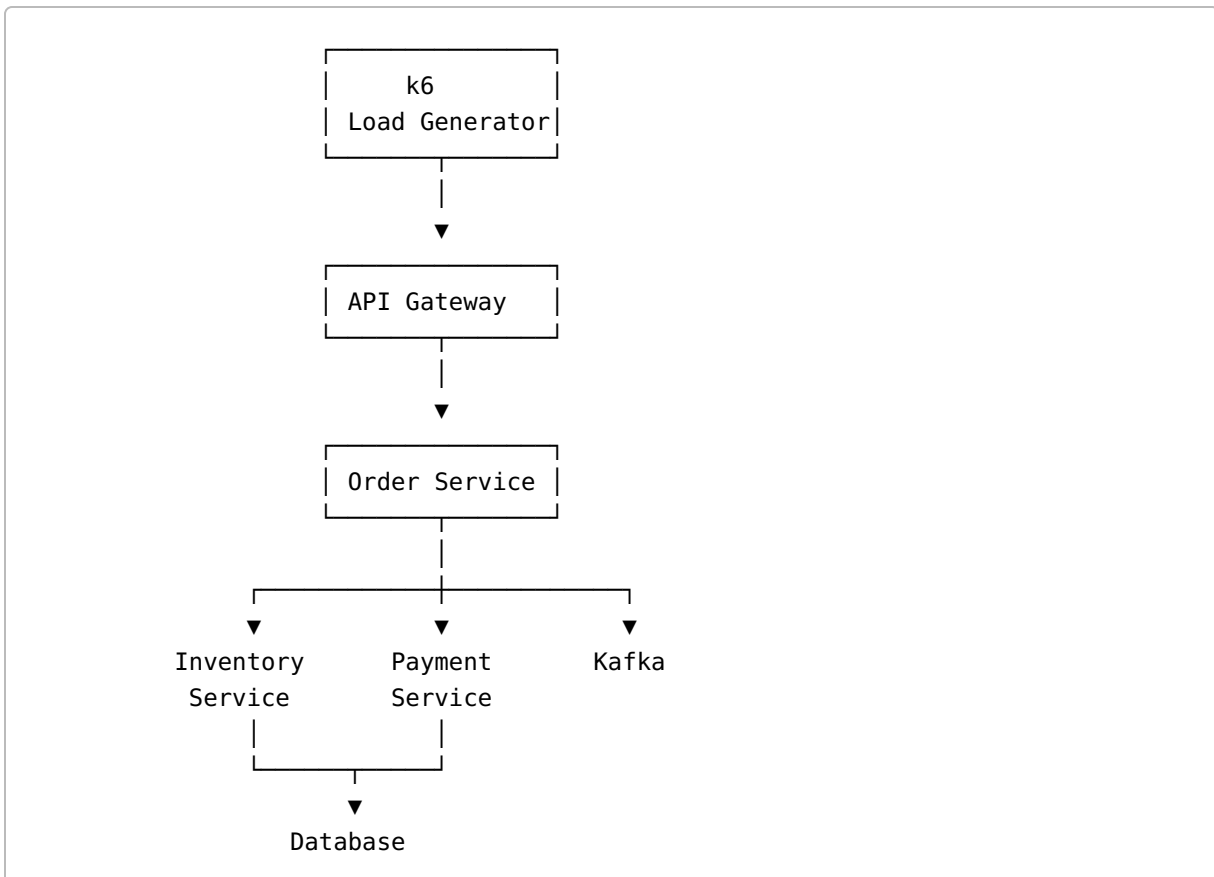
integration tests
small shell loops

These are useful.

But they do not answer an important question:

What happens when many clients access the system concurrently?

Consider our e-commerce platform:



The system may work perfectly with:

1 request

and still fail under:

100 concurrent requests

Performance testing helps us discover:

- latency problems;
 - throughput limits;
 - connection pool exhaustion;
 - CPU saturation;
 - database bottlenecks;
 - downstream failures;
 - queue buildup;
 - timeout problems;
 - retry amplification;
 - Circuit Breaker activation;
 - Bulkhead rejection;
 - capacity limits.
-

3. What Are We Actually Testing?

Performance testing is not simply:

"Send as many requests as possible."

A professional performance test asks a specific question.

For example:

Question 1

Can the system handle the expected normal traffic?

→ Load Test

Question 2

What happens when traffic becomes much higher than expected?

→ Stress Test

Question 3

What happens when traffic suddenly jumps?

→ Spike Test

Question 4

Does the endpoint work at all?

→ Smoke Test

Question 5

Where does the system become slow?

→ Bottleneck Analysis

4. k6

We will use:

Grafana k6

k6 is a performance-testing tool that allows us to describe workloads using JavaScript.

A very simple test looks like:

```
import http from 'k6/http';

export default function () {
  http.get('http://localhost:8080/api/v1/products');
}
```

When k6 runs this function repeatedly with multiple VUs, it generates concurrent traffic.

5. Virtual Users

A Virtual User, or VU, is a simulated client executing the test function.

For example:

```
1 VU
```

means approximately:

```
Client 1
  ↓
execute script
  ↓
request
  ↓
sleep
  ↓
repeat
```

With:

```
10 VUs
```

we have:

```
VU 1 ┌─┐
VU 2 ┌─┐
VU 3 ┌─┐
VU 4 ┌─┐
... ┌─┐ ──> System Under Test
VU 10 ┌─┐
```

But:

10 VUs does NOT necessarily mean 10 real human users.

A real user may:

```
open page
think
read
click
wait
scroll
read
click
```

A k6 VU may do:

```
request
sleep 1 second
```

```
request  
sleep 1 second  
request  
...
```

Therefore:

VUs are a workload-generation abstraction, not a literal count of human beings.

6. Iteration

An iteration is one complete execution of:

```
export default function () {  
  ...  
}
```

For example:

```
export default function () {  
  http.get('/products');  
  sleep(1);  
}
```

One execution of that function is:

1 iteration

If we have:

10 VUs

and each VU performs:

20 iterations

we have approximately:

200 iterations

The actual number depends on the workload model and execution time.

7. VUs and Throughput

Suppose:

10 VUs

and each iteration takes approximately:

1 second

A rough intuition is:

10 iterations/sec

But if each iteration takes:

5 seconds

the same 10 VUs may produce roughly:

2 iterations/sec

Therefore:

More VUs does not automatically mean a specific requests-per-second rate.

This is one reason k6 also provides arrival-rate executors when we want to control the rate of iteration starts independently of system response time.

8. Four Important Test Types

8.1 Smoke Test

Purpose:

Does the test work at all?

Example:

```
1 VU
10 seconds
```

We are checking:

- endpoint available;
 - authentication works;
 - response is correct;
 - script works;
 - thresholds work.
-

8.2 Load Test

Purpose:

Can the system handle expected traffic?

Example:

```
10 VUs
    ↓
20 VUs
    ↓
20 VUs sustained
    ↓
0 VUs
```

We are interested in:

- latency;
- throughput;
- errors;

- stability.
-

8.3 Stress Test

Purpose:

Where does the system stop behaving acceptably?

We intentionally exceed expected traffic.

Example:

```
20
↓
50
↓
100
↓
150
↓
200
```

We observe:

- latency degradation;
 - failures;
 - resource saturation;
 - resilience patterns;
 - recovery behavior.
-

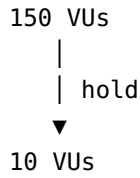
8.4 Spike Test

Purpose:

Can the system survive a sudden traffic increase?

Example:

```
10 VUs
|
| sudden jump
▼
```



This is different from gradually increasing traffic.

9. Test Type Decision Table

Question	Test
Does the script work?	Smoke
Can normal expected traffic be handled?	Load
Where does the system break?	Stress
Can the system survive sudden traffic?	Spike
Will a particular Bulkhead threshold activate?	Targeted stress
Where is the bottleneck?	Load + monitoring + tracing

10. Installing k6

Install k6 according to your operating system.

After installation:

```
k6 version
```

You should see the installed k6 version.

Then create:

```
session23/
```

Inside it:

```
session23/  
├─ smoke-test.js  
├─ load-test.js  
└─ stress-test.js
```

11. Your First k6 Test

Create:

```
smoke-test.js
```

Use:

```
import http from 'k6/http';  
import { check, sleep } from 'k6';  
  
export const options = {  
  vus: 1,  
  duration: '10s',  
  
  thresholds: {  
    http_req_duration: ['p(95)<500'],  
    http_req_failed: ['rate<0.01'],  
  },  
};  
  
export default function () {  
  
  const res = http.get(  
    'http://localhost:8080/api/v1/products'  
  );  
  
  check(res, {  
    'status is 200': (r) => r.status === 200,  
  });  
  
  sleep(1);  
}
```

Run:

```
k6 run smoke-test.js
```

12. Understanding the Script

Import HTTP

```
import http from 'k6/http';
```

This gives us HTTP functions such as:

```
http.get()  
http.post()  
http.put()  
http.delete()
```

Import check and sleep

```
import { check, sleep } from 'k6';
```

`check()` verifies an assertion.

`sleep()` pauses the VU.

13. The Options Object

```
export const options = {
```

contains the test configuration.

For example:

```
vus: 1,  
duration: '10s',
```

means:

```
1 VU
running for
10 seconds
```

14. Checks

This:

```
check(res, {
  'status is 200': (r) => r.status === 200,
});
```

means:

Check whether the response status is 200.

A failed check is recorded by k6.

But remember:

A check alone does not necessarily fail the entire test.

For that we use thresholds.

15. Thresholds

Example:

```
thresholds: {
  http_req_duration: [
    'p(95)<500'
  ],
  http_req_failed: [
    'rate<0.01'
  ],
}
```

```
},
```

means:

Latency

95% of HTTP requests
must complete in less than 500 ms

Errors

less than 1% of HTTP requests
may fail

Thresholds turn performance requirements into automated pass/fail conditions.

If a threshold is violated, k6 marks the test as failed and returns a non-zero exit code.

16. Percentiles

Do not rely only on average latency.

Suppose we have:

Average = 300 ms
P95 = 1200 ms
P99 = 3000 ms

The average looks acceptable.

But the tail latency is poor.

P50

50% of requests are faster than this value.

P95

95% of requests are faster than this value.

Therefore:

P95 = 500 ms

means approximately:

95% → ≤ 500 ms

5% → > 500 ms

P99

99% of requests are faster than this value.

The remaining 1% can represent a significant number of requests in a large system.

17. A Better Way to Think About Performance

Instead of asking:

"What is the average response time?"

ask:

How many requests?
At what rate?
For how long?
At what latency?
With what error rate?
Using how much CPU?
Using how much memory?
What happened downstream?

18. Test the Real Secured Platform

Our load test should not bypass:

API Gateway
Security

```
Service discovery
Resilience
Downstream calls
Database
Kafka
```

We want to test the real architecture.

Therefore the test should use a valid JWT.

For example:

```
TEST_JWT=<valid-token>
```

Then:

```
k6 run -e TEST_JWT="$TEST_JWT" load-test.js
```

Never hard-code real credentials into a committed k6 script.

19. Environment Variables

Instead of:

```
const baseUrl =
  'http://localhost:8080';
```

we can use:

```
const BASE_URL =
  __ENV.BASE_URL || 'http://localhost:8080';
```

Then:

```
k6 run
  -e BASE_URL=http://localhost:8080
  load-test.js
```

For the JWT:

```
const token = __ENV.TEST_JWT;
```

20. Docker Networking

If k6 runs directly on your machine:

```
http://localhost:8080
```

may be correct.

But if k6 runs inside Docker:

```
localhost
```

means:

```
the k6 container
```

not your API Gateway container.

If all services are on:

```
platform-net
```

use the Docker service name, for example:

```
const BASE_URL =  
  'http://api-gateway:8080';
```

The exact hostname depends on your Docker Compose service name.

21. Baseline Test

Before performing a stress test, establish a baseline.

Use:

1 VU

then:

5 VUs

then:

10 VUs

Record:

VUs	Requests/sec	P95	P99	Errors
1				
5				
10				

This gives us something to compare against.

22. The Staged Load Test

Create:

load-test.js

Use:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const BASE_URL =
  __ENV.BASE_URL || 'http://localhost:8080';

export const options = {
```

```

stages: [

  // Ramp-up
  { duration: '30s', target: 10 },

  // Expected load
  { duration: '30s', target: 20 },

  // Sustained load
  { duration: '2m', target: 20 },

  // Ramp-down
  { duration: '30s', target: 0 },

],

thresholds: {

  http_req_duration: [
    'p(95)<800',
    'p(99)<2000',
  ],

  http_req_failed: [
    'rate<0.05',
  ],

},

};

export default function () {

  const token = __ENV.TEST_JWT;

  const payload = JSON.stringify({

    productId: 'PROD-001',
    quantity: 1,
    amount: 100.00,

  });

  const params = {

    headers: {

      'Content-Type':
        'application/json',
    },
  },

```

```

        'Authorization':
            `Bearer ${token}`,

    },

};

const res = http.post(
    `${BASE_URL}/api/orders`,
    payload,
    params
);

check(res, {

    'status is successful':
        (r) => [200, 202].includes(r.status),

});

sleep(1);
}

```

Run:

```

k6 run
-e TEST_JWT="$TEST_JWT"
load-test.js

```

The `stages` shortcut represents a `ramping-vus` workload: k6 changes the target VU count according to the configured stages.

23. What Does This Workload Mean?

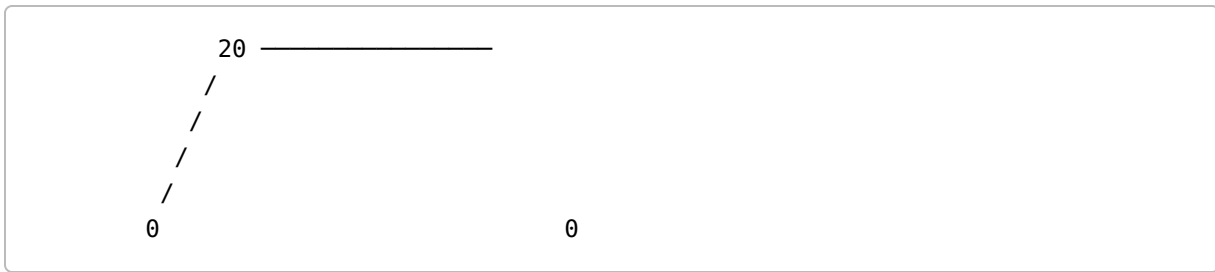
Our test:

```

30s → 10 VUs
30s → 20 VUs
2m → 20 VUs
30s → 0 VUs

```

represents:



Conceptually:

```
Ramp-up
  ↓
Expected traffic
  ↓
Stable period
  ↓
Ramp-down
```

A steady-state period is important because we want stable measurements rather than measuring only startup behavior. Grafana's k6 guidance similarly recommends a ramp-up, steady-state, and ramp-down profile for baseline testing.

24. Important Warning About POST /orders

Our test sends:

```
POST /api/orders
```

This changes system state.

Therefore:

Do not run this against production.

Use:

```
test database
test customer
test products
```

```
test payment environment
test Kafka topics
```

Also consider the Idempotency mechanism introduced in Session 22.

If the API supports:

```
Idempotency-Key
```

the test should use an appropriate test strategy rather than accidentally creating uncontrolled duplicate business operations.

25. Reading k6 Results

At the end of the test, look for:

```
http_req_duration
http_req_failed
iterations
vus
```

For example:

```
http_req_duration:
  avg=320ms
  p(90)=500ms
  p(95)=700ms
  p(99)=1.5s

http_req_failed:
  1.2%
```

Interpretation:

Average latency	= 320 ms
95th percentile	= 700 ms
99th percentile	= 1.5 s
Failure rate	= 1.2%

If our threshold is:

```
http_req_failed: ['rate<0.05']
```

then:

```
1.2% < 5%
```

so the error-rate threshold passes.

26. Throughput

Look at:

```
http_reqs
```

and:

```
http_reqs/s
```

Throughput tells us how much traffic the system actually processed.

For example:

```
150 requests/sec
```

is very different from:

```
20 requests/sec
```

even if both tests used:

```
50 VUs
```

This is why:

VU count alone is not a performance requirement.

27. The Stress Test

Now we intentionally exceed expected traffic.

Create:

```
stress-test.js
```

We can reuse the same request function and change the workload:

```
export const options = {  
  stages: [  
    { duration: '20s', target: 20 },  
    { duration: '30s', target: 50 },  
    { duration: '30s', target: 100 },  
    { duration: '30s', target: 150 },  
    { duration: '30s', target: 200 },  
    { duration: '30s', target: 0 },  
  ],  
  thresholds: {  
    // During this exploratory stress test,  
    // do not use strict business thresholds.  
  
    http_req_duration: [  
      'p(95)<5000',  
    ],  
  },  
};
```

Why are the thresholds intentionally looser?

Because this test is not primarily asking:

"Does the system meet the normal SLA?"

It is asking:

"What happens as we push the system toward its limits?"

28. Do Not Confuse Stress Testing With Failure Testing

A stress test is not:

make everything crash

We are interested in **how the system degrades**.

For example:

20 VUs
P95 = 400 ms
Errors = 0%

50 VUs
P95 = 600 ms
Errors = 0%

100 VUs
P95 = 1.2 s
Errors = 1%

150 VUs
P95 = 3.5 s
Errors = 8%

200 VUs
P95 = 8 s
Errors = 25%

This tells us much more than simply:

"the system failed."

29. The Resilience Experiment

Now we connect today's work with Sessions 4 and 5.

We previously configured:

```
Bulkhead  
TimeLimiter  
Circuit Breaker  
Retry
```

Today we ask:

What actually happens when the system is under pressure?

This is an experiment.

Do not assume the answer.

30. Before the Stress Test

Open separate terminals.

Terminal 1 — k6

```
k6 run  
-e TEST_JWT="$TEST_JWT"  
stress-test.js
```

Terminal 2 — Bulkhead

For example:

```
curl http://localhost:8082/actuator/bulkheads
```

Terminal 3 — Circuit Breaker

```
curl http://localhost:8082/actuator/circuitbreakers
```

Terminal 4 — Circuit Breaker events

```
curl http://localhost:8082/actuator/circuitbreakerevents
```

Resilience4j exposes Actuator endpoints for Circuit Breaker and Bulkhead information, as well as event endpoints.

31. What Are We Looking For?

We are looking for a sequence such as:

```
Traffic increases
  ↓
Latency increases
  ↓
Downstream becomes slow
  ↓
Timeouts / failures appear
  ↓
Bulkhead saturation/rejections
  ↓
Circuit Breaker records failures
  ↓
Circuit Breaker may OPEN
  ↓
Requests are rejected/fallbacks occur
```

But this sequence is a **hypothesis**, not a guaranteed universal execution order.

Your actual application configuration determines what you observe.

32. Why Bulkhead May Become Visible

Suppose:

```
maxConcurrentCalls: 10
```

This means the Bulkhead permits only a limited number of concurrent executions.

If the protected operation becomes slow:

```
Request 1 — running
Request 2 — running
...
Request 10 — running
```

then:

```
Request 11
Request 12
Request 13
...
```

may encounter a saturated Bulkhead depending on its configuration.

This is why simply increasing VUs is not enough.

We need:

```
high concurrency
+
slow enough protected operation
```

to make the behavior visible.

33. Controlled Slow-Down Experiment

If the platform contains a test-only endpoint or controlled downstream delay mechanism, introduce a safe delay.

For example:

```
Payment Service
  ↓
artificial 2-second delay
```

Then run:

```
10 VUs
20 VUs
50 VUs
100 VUs
```

Now concurrency accumulates more easily.

This produces a much better Bulkhead demonstration.

Do this only in a dedicated development/test environment.

34. Circuit Breaker Experiment

Suppose the Circuit Breaker is configured with:

```
minimumNumberOfCalls = 10
failureRateThreshold = 50%
```

Then the Circuit Breaker cannot simply open after one failed request.

It needs enough calls to evaluate the configured window.

For example:

```
Call 1 → failure
Call 2 → failure
...
Call 9 → failure
```

may still not be enough if:

```
minimumNumberOfCalls = 10
```

After enough calls:

```
10 calls
8 failures
```

gives:

80% failure rate

which may exceed:

50%

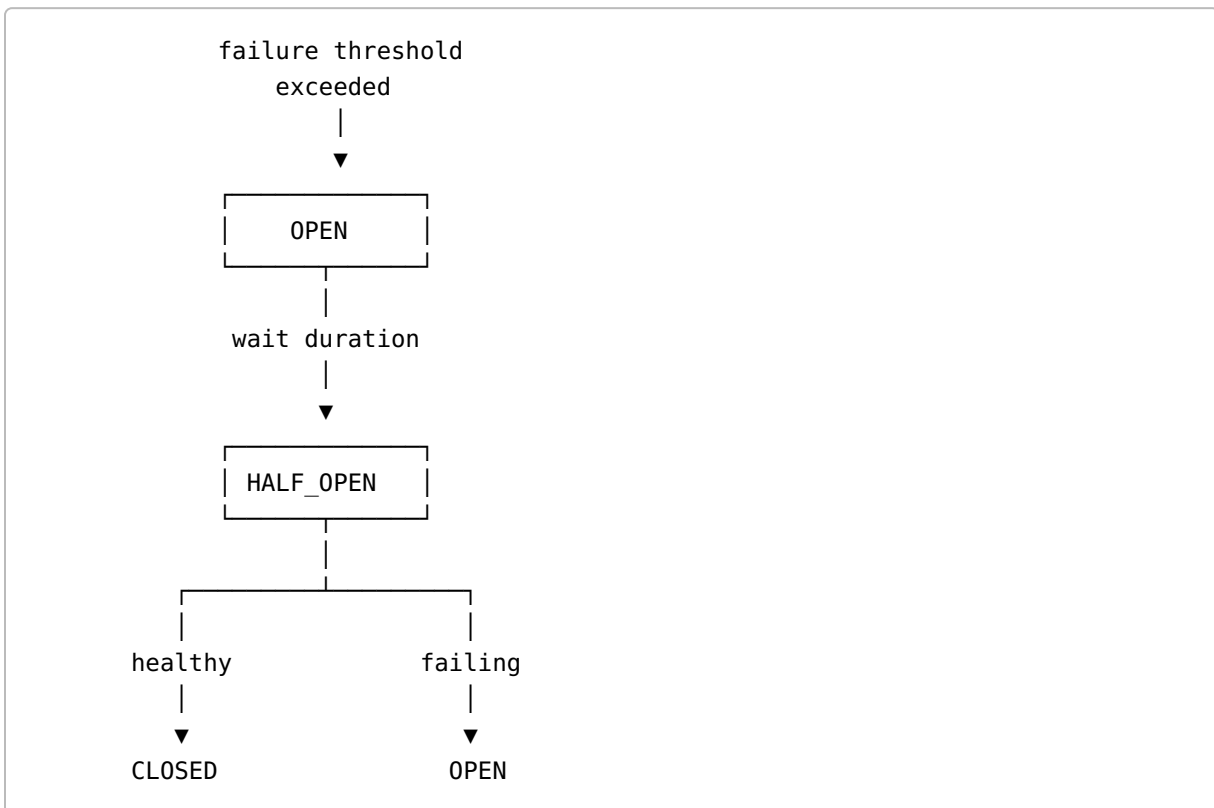
and cause:

CLOSED → OPEN

assuming the other Circuit Breaker conditions are satisfied.

35. Circuit Breaker State Machine

Remember:



The Circuit Breaker is therefore not simply:

working / broken

It is a state machine.

36. Do Not Forget Retry Amplification

Suppose k6 sends:

100 requests

and the service retries failed downstream calls:

3 attempts/request

The downstream service may receive substantially more attempts than the original client traffic.

Conceptually:

```
100 client requests
      ↓
100 primary calls
      ↓
failures
      ↓
retries
      ↓
additional calls
```

This is called retry amplification or retry storm behavior.

This is one reason performance testing should observe the entire chain rather than only the API Gateway.

37. Bottleneck Analysis

Now we reach the most important part of today's session.

Suppose k6 reports:

P95 = 2.5 seconds

That is a symptom.

It is not yet the root cause.

Ask:

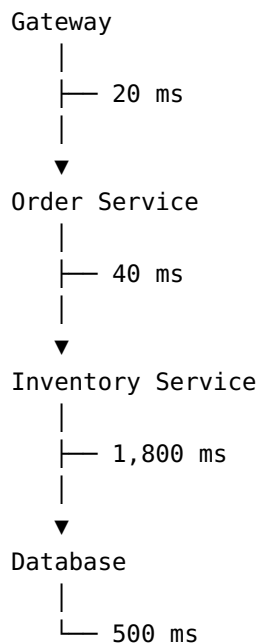
Where did the 2.5 seconds go?

38. Trace the Request

Use Zipkin from Session 17.

Find a representative slow request.

Conceptually:



Now we have evidence.

The problem is not simply:

"the API is slow."

We can say:

The majority of the request latency is occurring in the Inventory Service/database portion of the distributed request.

That is bottleneck analysis.

39. What to Monitor?

During the test, inspect:

Gateway

CPU
Memory
request rate
response time
connection count

Order Service

CPU
memory
JVM
thread activity
HTTP latency
database pool

Database

CPU
connections
slow queries
locks
I/O
connection pool

Kafka

```
producer rate  
consumer rate  
consumer lag
```

Resilience4j

```
Bulkhead saturation  
Circuit Breaker state  
failure rate  
slow calls  
rejected calls
```

k6

```
requests/sec  
P50  
P95  
P99  
errors  
VUs  
iterations
```

40. Common Bottleneck Patterns

Pattern 1 — CPU bottleneck

You see:

```
CPU ≈ 100%  
P95 increasing
```

Possible conclusion:

```
CPU saturation
```

Pattern 2 — Database bottleneck

You see:

```
Application CPU = 50%  
DB CPU = 100%  
P95 increasing  
DB connections saturated
```

Likely bottleneck:

```
Database
```

Pattern 3 — Connection pool bottleneck

You see:

```
Hikari active connections = maximum  
pending requests increasing
```

The database itself may not be the ultimate bottleneck.

The application may be waiting for a connection.

Pattern 4 — Downstream bottleneck

You see:

```
Order Service CPU = normal  
Order Service latency = high  
  
Zipkin:  
Order → Payment = 2.5 sec
```

The downstream Payment Service may be the bottleneck.

Pattern 5 — Retry amplification

You see:

```
traffic increases
  ↓
failures increase
  ↓
retries increase
  ↓
downstream traffic increases
  ↓
more failures
```

The resilience mechanism itself may be contributing to the load.

41. The Most Important Performance Principle

Never conclude:

"The application is slow."

Instead ask:

```
Which request?
Which service?
Which operation?
Which dependency?
Which resource?
Which threshold?
Which trace?
Which metric?
```

Performance engineering is evidence-based diagnosis.

42. A Simple Bottleneck Investigation Workflow

Use this sequence:

1. Detect
↓
2. Measure
↓
3. Locate
↓
4. Explain
↓
5. Change
↓
6. Re-test

Example:

P95 increased
↓
Zipkin shows Payment slow
↓
Payment DB connection pool saturated
↓
Increase pool? Optimize query?
↓
Run test again
↓
Compare results

43. Performance Test Results Table

After every major run, record:

Test	VUs	Duration	Req/s	P50	P95	P99	Error %	Main Observation
Smoke	1	10s						
Baseline	5	2m						
Load	20	2m						
Stress	50	30s						
Stress	100	30s						
Stress	150	30s						
Stress	200	30s						

Do not record only:

PASS
FAIL

Record what happened.

44. Performance Testing Is Experimental

Suppose your prediction is:

Bulkhead will become visible first.

Record it.

Then execute the test.

Maybe the result is:

Circuit Breaker changed state first.

That is not a failed experiment.

It is a useful result.

Ask:

Why?

Maybe:

- downstream failures happened first;
- the Circuit Breaker was configured differently;
- Bulkhead concurrency was never saturated;
- requests completed too quickly;
- TimeLimiter rejected calls;
- Retry changed the observed behavior.

The goal is to understand the system.

45. Advanced k6 Concept — Arrival Rate

So far we have used:

VUs

But sometimes the requirement is:

Send approximately 50 requests per second.

That is a different requirement.

k6 provides arrival-rate executors.

Example:

```
export const options = {  
  scenarios: {  
    api_load: {  
      executor: 'constant-arrival-rate',  
      rate: 50,  
      timeUnit: '1s',  
      duration: '1m',  
      preAllocatedVUs: 50,  
    },  
  },  
};
```

This means we want approximately:

50 iterations / second

rather than simply:

50 VUs

k6 documents arrival-rate executors as an open workload model where iterations start independently of the system's response time.

For today's core lab, use `ramping-vus` first. Treat arrival-rate testing as the professional extension.

46. Closed Model vs Open Model

VU-based workload

Conceptually:

```
VU sends request
      ↓
waits for response
      ↓
continues
```

This is a closed-model style of workload.

Arrival-rate workload

Conceptually:

```
Start request
Start request
Start request
Start request
...
```

at a controlled rate.

The system's response time does not determine the desired arrival rate in the same way.

This distinction becomes important when doing serious capacity testing.

47. Why a Load Test Can Lie

Suppose:

```
k6 machine CPU = 100%
```

while:

```
microservices CPU = 40%
```

The bottleneck may be:

```
load generator
```

not the application.

Therefore always ask:

Can my load generator generate the requested workload without becoming the bottleneck?

For larger tests, distributed or cloud-based load generation may be required.

48. Performance Test Design Rules

Rule 1

Always establish a baseline.

Rule 2

Change one important variable at a time.

Rule 3

Use realistic data.

Rule 4

Warm up the application.

Rule 5

Do not test production accidentally.

Rule 6

Monitor the load generator.

Rule 7

Monitor the database.

Rule 8

Monitor downstream services.

Rule 9

Use traces to investigate latency.

Rule 10

Repeat tests before drawing conclusions.

49. Lab Exercise

Lab 23 — Find the Performance Limit

Objective

Determine how the platform behaves as traffic increases.

Step 1 — Verify the platform

Confirm:

API Gateway	UP
Order Service	UP
Payment Service	UP
Inventory Service	UP
Database	UP

Kafka	UP
Zipkin	UP

Step 2 — Run smoke test

```
k6 run smoke-test.js
```

Expected:

```
No major errors
```

Step 3 — Run baseline

Use:

```
5 VUs
```

Record:

```
requests/sec  
P50  
P95  
P99  
error rate
```

Step 4 — Run normal load

Use:

```
20 VUs
```

Run for approximately:

2 minutes

Record the results.

Step 5 — Run stress test

Increase gradually:

50
100
150
200

Do not jump immediately to a huge number.

At each stage record:

P95
P99
error rate
throughput
CPU
memory
database
Bulkhead
Circuit Breaker

Step 6 — Observe resilience

Determine:

Which resilience mechanism showed evidence first?

Do not assume the answer.

Step 7 — Find a slow trace

Use Zipkin.

Find one request from the period where latency increased.

Identify:

```
slowest service
slowest span
```

Step 8 — Identify the bottleneck

Write:

The primary bottleneck appears to be _____.

Evidence:

1. _____
2. _____
3. _____

50. Lab Report

Submit the following table:

Item	Result
Baseline VUs	
Maximum tested VUs	
Maximum sustainable throughput	
Baseline P95	
Stress P95	
Baseline error rate	
Stress error rate	

Item	Result
First visible resilience behavior	
Circuit Breaker state	
Bulkhead observation	
Slowest service	
Suspected bottleneck	
Evidence	

51. Questions to Answer

Q1

What is a VU?

Q2

Why is:

100 VUs

not equivalent to:

100 real human users?

Q3

What is the difference between:

check()

and:

threshold

?

Q4

What does:

$p(95) < 800$

mean?

Q5

What is the difference between load and stress testing?

Q6

Why do we need a baseline?

Q7

Why might 150 VUs fail to trigger a Bulkhead?

Q8

What conditions are required before a Circuit Breaker can open?

Q9

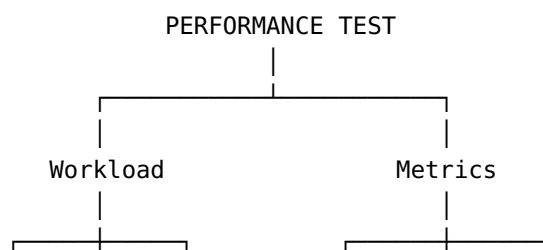
Why should we inspect Zipkin when P95 increases?

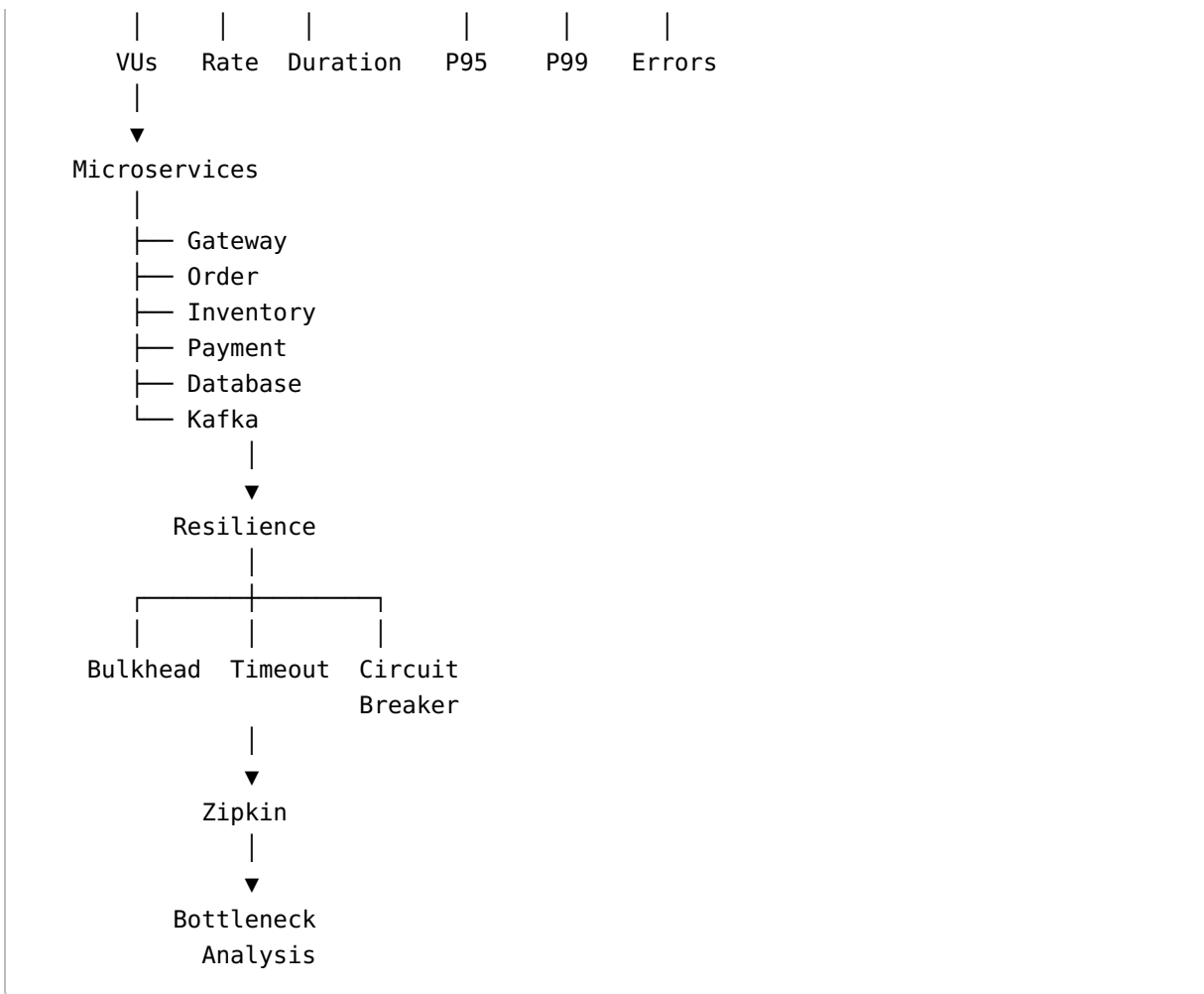
Q10

How can the load generator itself become the bottleneck?

52. Expected Mental Model

After this session, you should think about performance like this:





53. The Key Lesson

The purpose of today's session is **not**:

"Learn k6 commands."

The real objective is:

Learn how to experimentally determine how a distributed system behaves under increasing load.

We start with:

configuration

Then create:

real traffic

Then observe:

metrics

Then correlate:

metrics + traces + resilience state

Then determine:

bottleneck

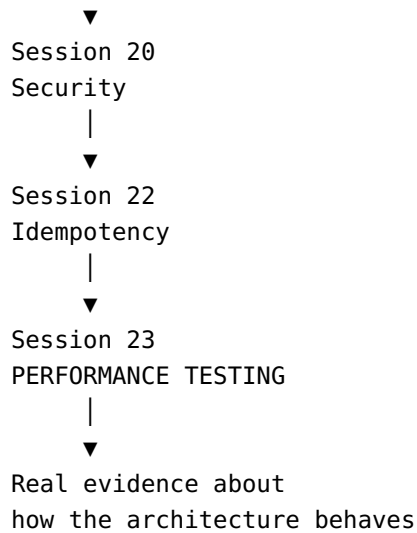
Then change the system and test again.

That is performance engineering.

54. Connection to Previous Sessions

Today's session closes an important loop:

```
Session 4–5  
Resilience  
  |  
  ▼  
Session 13  
Quality Gates  
  |  
  ▼  
Session 16  
Autoscaling  
  |  
  ▼  
Session 17  
Distributed Tracing  
  |
```



The architecture is no longer only something we configured.

Today we **put it under pressure and observe what actually happens.**

55. Professional Takeaway

A production engineer should never say:

"The service can handle 200 users."

A better statement is:

"Under the tested workload profile, the platform sustained approximately X requests/sec at Y VUs, with P95 latency of Z ms and an error rate of N%. Beyond this level, latency/error behavior changed and the evidence pointed to _____ as the primary bottleneck."

That is the difference between:

"I tested it."

and:

"I have evidence about its capacity and failure behavior."